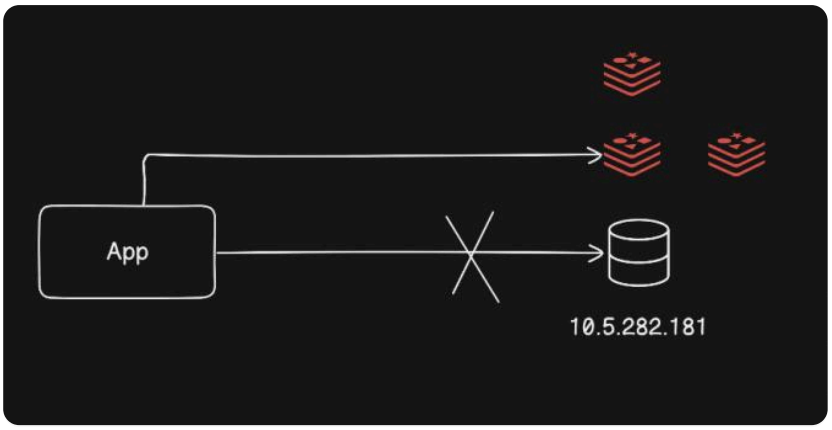
Exception Handling

04 July 2026 09:09

In real applications problems will happen.

`User may enter wrong input` `File may not exist` `Database may be down`




```
Exception in thread "main" java.lang.ArithmeticException: / by zero at Test.main(Test.java:9)
```

1. The program stopped at `a/b`
2. The next line did not execute
3. This runtime problem is called **exception**

What is Exception ?

> An exception is an unwanted or unexpected event that happens during the program execution and disturbs the normal flow of the program.

Exception = runtime problem in the program

Error

Usually represents a serious problem that app code normally cannot handle

Exception

Represents a problem that app code can handle.

Note: As backend developer, we mainly handle **exceptions**, not errors.

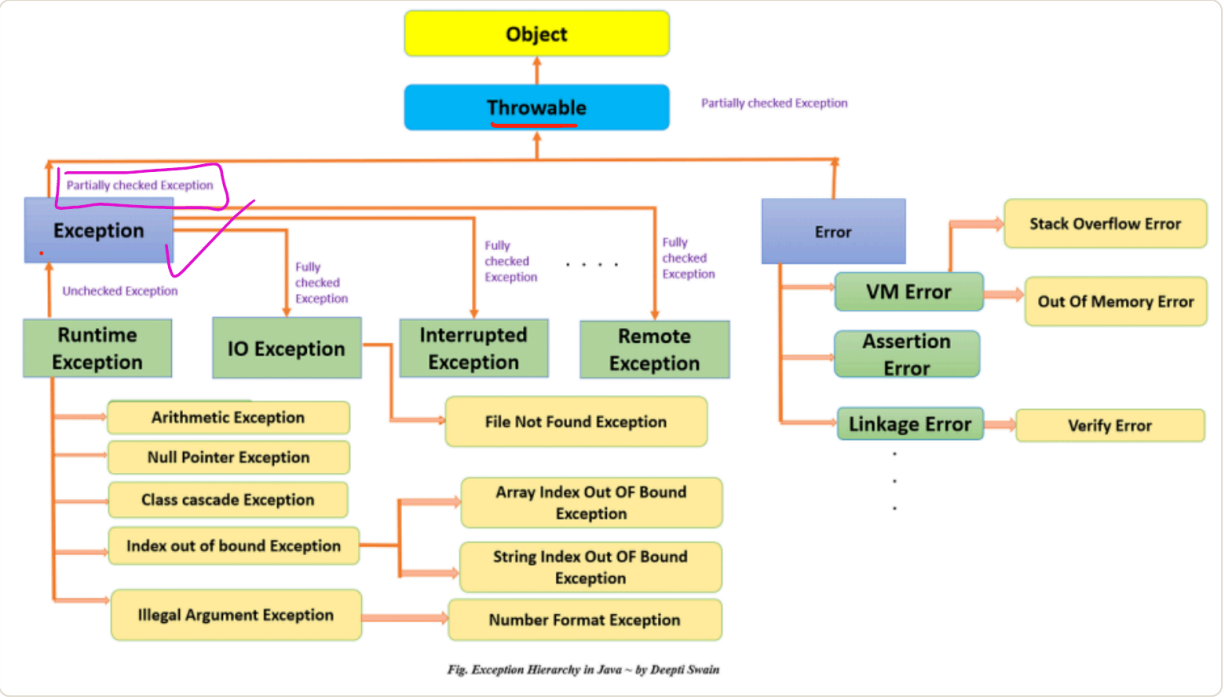


Fig. Exception Hierarchy in Java ~ by Deepti Swain

Types of exception

1. Checked Exception

Checked exceptions are those that the compiler forces you to handle explicitly.

Eg: `IOException` , `SQLException`

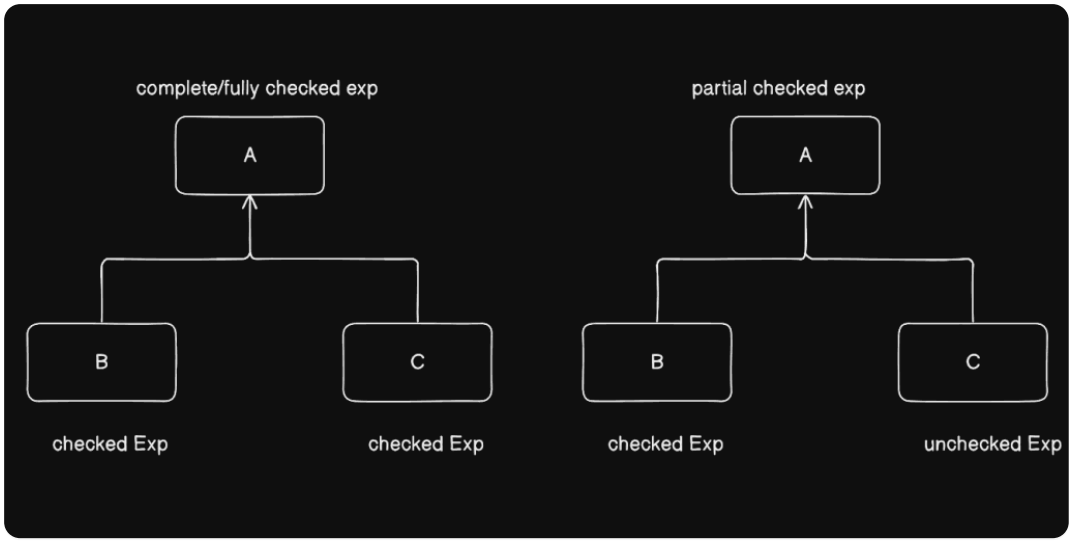
Handling checked exception is done by using **try-catch** block

2. Unchecked Exception

In Unchecked Exception compiler will not force you to handle explicitly but they are thrown at runtime.

Eg: `NullPointerException` , `ArrayIndexOutOfBoundsException`

> **Note:** `RuntimeException` and all its child classes, `Error` and its child classes are **unchecked**, and all remaining are considered as **checked** exceptions



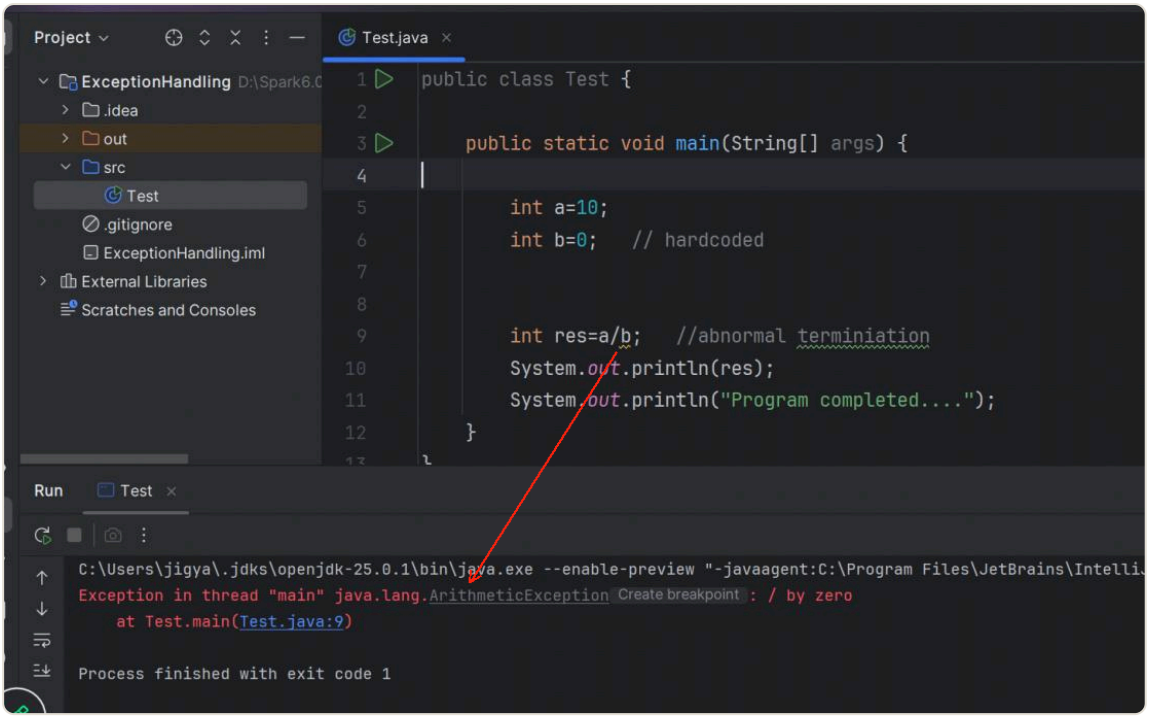
Compile Time Error Vs Runtime Exception

Compile time error:

This happens before program runs.

```
1  public class Test {
2
3      public static void main(String[] args) {
4
5          int age="twenty";
6      }
7  }
8
```

Runtime Exception:



Why do we need Exception handling ?

Without exception handling:

- Program stops suddenly
- User sees technical error
- Remaining code does not execute
- Application may give bad experience

With exception handling

- Program can show meaningful message
- Remaining code can continue
- Developer can log the real problem

Real-world examples:

Suppose user enters wrong password

Application should not crash, it should show:

Invalid username or password

Suppose Database is down.

Application should not expose full java error to user. It should return a proper response like:

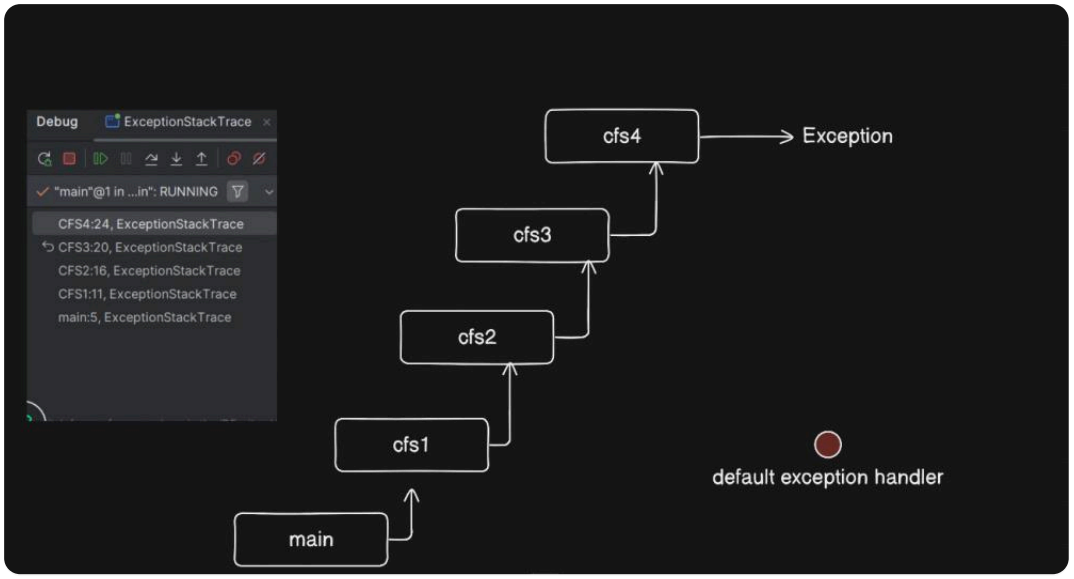
"Service temporarily unavailable"

Exception handling keywords

Java mainly gives these keywords:

`1. try-catch` `2. finally` `3. throw` `4. throws`

Keyword	Description
`try`	Write risky code
`catch`	Handle exception
`finally`	Always execute cleanup code
`throw`	Manually create/raise exception
`throws`	Delegate responsibility



Various methods to print information

`printStackTrace()`

This method prints exception information in following format:

```
Name of the exception: description of exception
stack trace
```

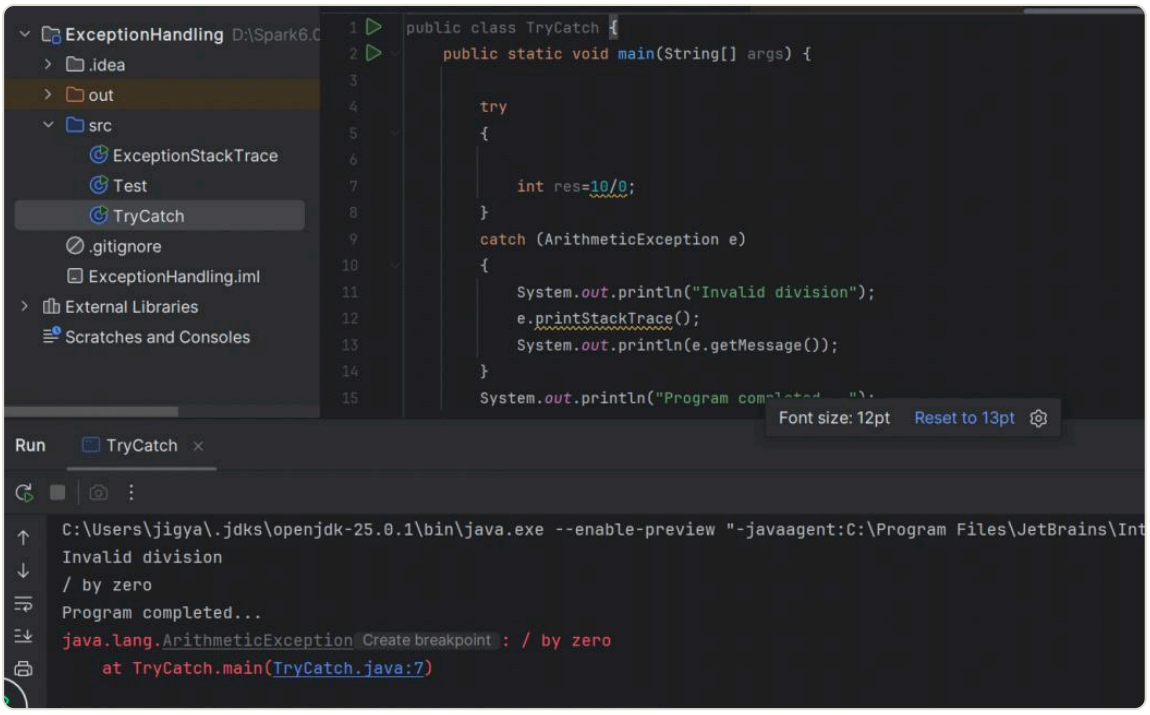
`toString()`

This method prints exception information in the following format:

```
Name of the exception: Description of exception
```

`getMessage()`

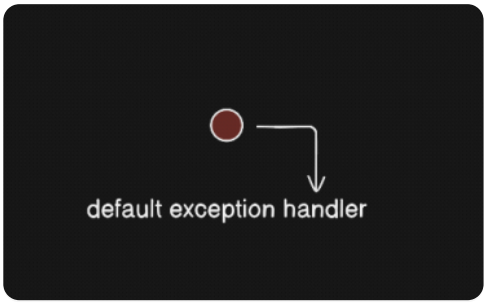
This method returns only description of the exception



Flow of try-catch

Finally

> **Finally** block contains code that should execute whether exception occurs or not.



throw:

> `throw` is used to manually throw an exception

Custom exception

`V.V.V.V.V.V.V.IMP` **How to create custom exception**